

AI(H) 期末复习讲义

从 AI Notes、Labs、Projects、Review、样卷与 Logic/Bayes 题整理

Yanqiao Chen

2026 年 6 月 8 日

目录

1	考点地图与复习策略	4
1.1	高概率题型	4
1.2	Labs 与 Projects 对应关系	4
2	Search 与 Optimization	5
2.1	搜索问题定义	5
2.2	BFS, UCS, Greedy, A*	5
2.3	样卷题: 求边权 c 的范围	6
2.4	Beyond Classical Search 与 Project1	6
3	Adversarial Search	6
3.1	Minimax	6
3.2	Alpha-Beta Pruning	6
4	CSP 与 AC-3	7
4.1	CSP 三元组	7
4.2	AC-3	7
4.3	AC-3 例题模板	7
5	Logic, CNF, DPLL 与 Resolution	8
5.1	必背等价式	8
5.2	CNF 转换模板	8
5.3	Logic and Bayesian Questions: 排球队建模	8
5.4	FOL Resolution	9

6 Machine Learning 基本原则	9
7 Perceptron 与 Logistic Regression	10
7.1 Perceptron	10
7.2 Logistic Regression	10
8 SVM	10
8.1 Hard Margin	10
8.2 Soft Margin	10
8.3 Kernel Trick	11
9 Naive Bayes 与 Bayesian Networks	11
9.1 Naive Bayes	11
9.2 Bayesian Network	11
9.3 Bayes Rule 例题	12
9.4 Sampling	12
10 Evaluation, Tree 与 Generalization	13
10.1 Confusion Matrix	13
10.2 Generalization 与 Tuning	13
10.3 Decision Tree	14
11 Unsupervised Learning	14
11.1 K-means	14
11.2 PCA	15
11.3 LLE	15
12 Recommender Systems 与 Knowledge Graphs	16
12.1 推荐系统基础	16
12.2 Pearson Similarity	16
12.3 Matrix Factorization	17
12.4 Knowledge Graph	17
12.4.1 KG Construction	18
12.4.2 Knowledge Graph Completion	18
12.4.3 KG-Based Recommender System	19

13 PPT 强调的概念问答	19
13.1 Search 与 Problem-Specific Search	19
13.2 Beyond Classical Search 与 Evolutionary Search	20
13.3 Machine Learning 与 Evaluation	21
13.4 SVM, Neural Network 与 Unsupervised Learning	21
13.5 Logic, FOL, BN 与 KG	22
14 Genetic Algorithm 与 Fair Comparison	23
15 Reinforcement Learning	23
16 考前最后检查清单	24

1 考点地图与复习策略

这份讲义把 AI Notes.md 的主线整理成可打印的 LaTeX 版本，并把前面 labs 和 projects 的内容合并到同一套考试视角里：先背公式和模板，再看典型题怎么落笔。

1.1 高概率题型

优先级	主题	必须会做什么
高	Search	画 BFS/UCS/A* 搜索树；算 g, h, f ；判断 heuristic 是否 admissible/consistent；求边权 c 的范围。
高	Minimax/Alpha-Beta	自底向上填 minimax 值；维护 α, β ；标出剪枝分支和最终路径。
高	CSP/AC-3	写 CSP 三元组；手跑 AC-3 domain pruning；说明 $O(ed^3)$ 。
高	Logic/CNF/DPLL	自然语言建模；消去 $\rightarrow, \leftrightarrow$ ；转 CNF；用 unit propagation 或 resolution 推导。
高	Logistic/Perceptron/SVM	写更新公式；推 cross-entropy 梯度；写 hard/soft-margin SVM primal；解释 C 和 kernel。
高	Naive Bayes/BN	算先验、条件概率、后验；解释独立性；会 prior/rejection/likelihood weighting/Gibbs sampling。
中高	ML Evaluation	confusion matrix、ROC/AUC、cross-validation、bootstrap、hyperparameter tuning。
中高	Tree/K-means/PCA/LLE	entropy/IG、聚类迭代、PCA 投影、LLE 局部重构流程。
中高	Recommender/KG	Pearson similarity、matrix factorization、RDF triples、NER、relation extraction、completion。
中高	GA/RL	GA 公平实验设计；Bellman equation；Q-learning update。

1.2 Labs 与 Projects 对应关系

材料	核心内容	考试中怎么用
Lab1	Python/Numpy 基础	不太直接考，主要支撑矩阵、向量、数据处理。
Lab2	A* heuristic search	搜索树、frontier、 $g + h$ 、admissible heuristic。
Lab3	N-Queens	CSP 表示、约束、backtracking、局部搜索。
Lab4	CSP practice	AC-3、MRV、Degree、LCV。
Lab5	Adversarial search	Minimax 与 alpha-beta pruning。
Lab6-7	Credit-card fraud, MLP/SVM	分类、混淆矩阵、precision/recall/F1、SVM。

Lab8	Perceptron, SVM, decision tree	感知机更新、margin、entropy/IG/pruning。
Lab9	K-means	assignment/update、初始化敏感、outlier 敏感。
Lab10	PCA/LLE	线性降维 vs 非线性流形学习。
Lab11	Matrix factorization	推荐系统、 $R \approx PQ^T$ 、embedding。
Project1	Information Exposure Maximization	IC diffusion、Monte Carlo、greedy heuristic、IMRank、GA、复杂度与实验公平性。
Project2	Classification/Retrieval/Feature Selection	softmax regression、distance metric、PCA/LLE、wrapper/filter feature selection、generalization。
Project3	KG-based recommender	KG triples、content-based + collaborative filtering、MF、BCE+BPR、AUC/nDCG。

答题句式: 如果题目让“结合课程项目说明”，优先把算法写成 Representation + Algorithm + Evaluation: 数据怎么表示，怎么搜索/优化，用什么 metric 评价。

2 Search 与 Optimization

2.1 搜索问题定义

一个 classical search problem 通常包含:

Initial state, $Actions(s)$, $Result(s, a)$, $GoalTest(s)$, $PathCost$.

常用符号:

b = branching factor, d = shallowest goal depth, m = max depth, C^* = optimal path cost.

2.2 BFS, UCS, Greedy, A*

算法	Evaluation	Complete	Optimal
BFS	depth	若 b 有限	每步代价相同
UCS	$f(n) = g(n)$	若 step cost > 0	是
Greedy	$f(n) = h(n)$	不保证	不保证
A*	$f(n) = g(n) + h(n)$	通常是	tree: admissible; graph: consistent

Admissible:

$$h(n) \leq h^*(n), \quad \forall n.$$

启发函数不能高估到 goal 的真实最小代价。

Consistent:

$$h(n) \leq c(n, a, n') + h(n'), \quad \forall (n, n').$$

Consistent 一定 admissible; A* graph search 通常要求 consistent 才能保证 optimal。

2.3 样卷题：求边权 c 的范围

若每条边代价都是 c ，从 n 到 goal 的最少边数为 $dist_{edges}(n, G)$ ，则

$$h^*(n) = c \cdot dist_{edges}(n, G).$$

Admissible 要求

$$h(n) \leq c \cdot dist_{edges}(n, G), \quad c \geq \frac{h(n)}{dist_{edges}(n, G)}.$$

所以最终答案：

$$c \geq \max_{n \neq G} \frac{h(n)}{dist_{edges}(n, G)}.$$

还要检查 $h(G) = 0$ 。如果 $h(G) > 0$ ，任何正 c 都不 **admissible**。

2.4 Beyond Classical Search 与 Project1

Beyond classical search 常直接在 solution space 搜索。通用框架：

$$x^{(0)} \rightarrow f(x^{(0)}) \rightarrow x' \rightarrow f(x') \rightarrow \text{accept/reject}.$$

常见 operator: binary flip、swap、Gaussian perturbation。

Project1 的 IEMP 是典型组合优化：

$$\max_{S_1, S_2} \mathbb{E}[|V| - |R_1 \oplus R_2|], \quad |S_1| + |S_2| \leq k, \quad S_1 \cap S_2 = \emptyset.$$

其中 R_1, R_2 是 Independent Cascade diffusion 后被两个 campaign reached 的节点集合。因为精确期望很难算，所以用 Monte Carlo：

$$\hat{\Phi}(S_1, S_2) = \frac{1}{T} \sum_{t=1}^T (|V| - |R_1^{(t)} \oplus R_2^{(t)}|).$$

答题句式: Project1 可以写成: candidate seed sets 是状态; MC evaluator 是 objective estimator; greedy/GA 是 search algorithm; final score 是 expected balanced exposure 的 Monte Carlo estimate。

3 Adversarial Search

3.1 Minimax

适用场景: deterministic, two-player, zero-sum, perfect-information games。

$$V_{\text{MAX}}(n) = \max_{c \in \text{Children}(n)} V(c), \quad V_{\text{MIN}}(n) = \min_{c \in \text{Children}(n)} V(c).$$

3.2 Alpha-Beta Pruning

α = MAX 当前路径能保证的最好值, β = MIN 当前路径能保证的最好值。

剪枝条件:

$$\boxed{\alpha \geq \beta}.$$

做题步骤:

1. 从左到右访问叶子，除非题目指定顺序。
2. MAX 节点初始 $v = -\infty$ ，更新 $v = \max(v, child)$ 和 $\alpha = \max(\alpha, v)$ 。
3. MIN 节点初始 $v = +\infty$ ，更新 $v = \min(v, child)$ 和 $\beta = \min(\beta, v)$ 。
4. 一旦 $\alpha \geq \beta$ ，剩余分支剪掉；被剪枝节点可写 NA。
5. 最终路径从 root 开始，每层选等于父节点 minimax value 的 child。

Alpha-beta pruning 不改变 minimax value，只减少访问节点数；剪枝多少依赖访问顺序。

4 CSP 与 AC-3

4.1 CSP 三元组

$$CSP = (X, D, C),$$

其中 X 是变量集合， D 是 domains， C 是 constraints。

例:

$$X = \{X, Y, Z\}, \quad D_X = D_Y = D_Z = \{1, 2, 3\}, \quad C = \{X < Y, Y < Z\}.$$

4.2 AC-3

Arc (X_i, X_j) consistent 的含义:

$$\forall x \in D_i, \exists y \in D_j \quad \text{s.t.} \quad \text{constraint}(X_i = x, X_j = y) \text{ holds.}$$

复杂度:

$$\boxed{O(ed^3)}.$$

原因: 一次 REVISE 最多 $O(d^2)$; 每条 arc 最多因删除 domain value 被重新加入 $O(d)$ 次。

4.3 AC-3 例题模板

对 $D_X = D_Y = D_Z = \{1, 2, 3\}, C = \{X < Y, Y < Z\}$:

1. Revise (X, Y) : 删 $X = 3$, $D_X = \{1, 2\}$ 。
2. Revise (Y, X) : 删 $Y = 1$, $D_Y = \{2, 3\}$ 。
3. Revise (Y, Z) : 删 $Y = 3$, $D_Y = \{2\}$ 。
4. 因 D_Y 变化，重查相关 arcs。

5. Revise (Z, Y) : 只剩 $Z = 3$, $D_Z = \{3\}$ 。

6. Revise (X, Y) : 只剩 $X = 1$, $D_X = \{1\}$ 。

最终:

$$D_X = \{1\}, \quad D_Y = \{2\}, \quad D_Z = \{3\}.$$

Backtracking 启发式: MRV 选合法值最少变量; Degree 打平时选约束最多变量; LCV 选限制别人最少的值。

5 Logic, CNF, DPLL 与 Resolution

5.1 必背等价式

$$p \rightarrow q \equiv \neg p \vee q, \quad p \leftrightarrow q \equiv (\neg p \vee q) \wedge (\neg q \vee p).$$

$$\neg(p \wedge q) \equiv \neg p \vee \neg q, \quad \neg(p \vee q) \equiv \neg p \wedge \neg q.$$

$$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r).$$

CNF 是 AND of OR-clauses, 例如

$$(a \vee b \vee \neg c) \wedge (\neg a \vee d).$$

5.2 CNF 转换模板

1. 消去 $\leftrightarrow$ 和 $\rightarrow$ 。
2. 用 De Morgan 把 $\neg$ 推到 atom 前。
3. 用 distributive law 把 OR 分配到 AND 上。
4. 化简重复项、恒真项和吸收项。

样卷公式:

$$\begin{aligned} p \rightarrow \neg(p \vee q) &\equiv \neg p \vee \neg(p \vee q) \\ &\equiv \neg p \vee (\neg p \wedge \neg q) \\ &\equiv \neg p. \end{aligned}$$

5.3 Logic and Bayesian Questions: 排球队建模

令 x_i 表示 player i plays。

$$x_4 \leftrightarrow x_6, \quad \neg x_3 \leftrightarrow \neg x_1 \equiv x_3 \leftrightarrow x_1.$$

$$x_3 \oplus x_6 \equiv (x_3 \vee x_6) \wedge (\neg x_3 \vee \neg x_6).$$

$$(x_9 \wedge x_{12}) \rightarrow x_4 \equiv \neg x_9 \vee \neg x_{12} \vee x_4.$$

加 evidence: x_1, x_{12} 。

完整 CNF:

$$(\neg x_4 \vee x_6) \wedge (\neg x_6 \vee x_4) \wedge (\neg x_3 \vee x_1) \wedge (\neg x_1 \vee x_3) \\ \wedge (x_3 \vee x_6) \wedge (\neg x_3 \vee \neg x_6) \wedge (\neg x_9 \vee \neg x_{12} \vee x_4) \wedge x_1 \wedge x_{12}.$$

DPLL unit propagation:

$$x_1, x_{12} \Rightarrow x_3 \Rightarrow \neg x_6 \Rightarrow \neg x_4 \Rightarrow \neg x_9.$$

5.4 FOL Resolution

UI: $\forall x P(x) \Rightarrow P(a)$ 。EI: $\exists x P(x) \Rightarrow P(k)$, 其中 k 必须是新常量。

Unification 找 substitution。例如

$$Knows(John, x), Knows(y, Mary)$$

可用 $\{x/Mary, y/John\}$ 统一。

证明 $Happy(z)$ 的典型 CNF:

$$\neg PassAI(x) \vee \neg Prize(x) \vee Happy(x), \\ \neg Willing(x) \vee PassAI(x), \quad \neg Lucky(x) \vee PassAI(x), \quad \neg Lucky(x) \vee Prize(x), \\ \neg Willing(z), \quad Lucky(z), \quad \neg Happy(z).$$

Resolution:

$$Lucky(z) \Rightarrow PassAI(z), \quad Lucky(z) \Rightarrow Prize(z), \\ PassAI(z), Prize(z) \Rightarrow Happy(z),$$

与 $\neg Happy(z)$ 得 empty clause $\square$ 。

6 Machine Learning 基本原则

Learning = search in hypothesis/model space.

三件核心事:

$$\text{Representation} + \text{Algorithm} + \text{Evaluation} = \text{Model}.$$

Overfitting: training performance 好但 unseen data 差。Occam's Razor: 在解释数据的前提下模型尽量简单。

MAP 分类:

$$\hat{y} = \arg \max_{\omega_i} P(\omega_i | x) = \arg \max_{\omega_i} p(x | \omega_i) P(\omega_i),$$

因为 $p(x)$ 对所有类别相同。

Project2 的三任务对应:

- Classification: 输入 256 维 feature, 输出 10-class label。
- Retrieval: 给 query, 从 repository 找 top- k similar items。
- Feature selection: 选不超过 30 个 feature, 使固定 classifier accuracy 尽量高。

7 Perceptron 与 Logistic Regression

7.1 Perceptron

若 $y \in \{-1, +1\}$:

$$\hat{y} = \text{sign}(w^T x + b).$$

分类错误时:

$$w \leftarrow w + \eta y x, \quad b \leftarrow b + \eta y.$$

若 $y \in \{0, 1\}$, 常写为

$$w \leftarrow w + \eta(y - \hat{y})x, \quad b \leftarrow b + \eta(y - \hat{y}).$$

7.2 Logistic Regression

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad h_w(x) = \sigma(w^T x + b).$$

Binary cross-entropy:

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y_i \log h_i + (1 - y_i) \log(1 - h_i)].$$

关键梯度:

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^m (h_i - y_i) x_i, \quad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (h_i - y_i).$$

更新:

$$w \leftarrow w - \alpha \frac{1}{m} \sum_i (h_i - y_i) x_i, \quad b \leftarrow b - \alpha \frac{1}{m} \sum_i (h_i - y_i).$$

答题句式: 推导题写: 令 $z_i = w^T x_i + b, h_i = \sigma(z_i)$, cross-entropy 对 z_i 的导数是 $h_i - y_i$, 再链式法则乘 x_i 或 1。

8 SVM

8.1 Hard Margin

标签 $y_i \in \{-1, +1\}$:

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1.$$

Margin width:

$$\frac{2}{\|w\|}.$$

8.2 Soft Margin

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

$$\text{s.t. } y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

等价 hinge loss:

$$\min_{w,b} \frac{1}{2} \|w\|^2 C \sum_i \max(0, 1 - y_i(w^T x_i + b)).$$

C 大: 更重视训练错误, regularization 弱; C 小: 允许更多 violation, regularization 强。

8.3 Kernel Trick

$$K(x, z) = \phi(x)^T \phi(z).$$

常见 kernel:

$$K(x, z) = x^T z, \quad K(x, z) = (x^T z + c)^d, \quad K(x, z) = \exp(-\gamma \|x - z\|^2).$$

分别对应 linear、polynomial、RBF。

9 Naive Bayes 与 Bayesian Networks

9.1 Naive Bayes

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)}, \quad \hat{y} = \arg \max_y P(y)P(x | y).$$

Naive assumption:

$$P(x_1, \dots, x_n | y) = \prod_{j=1}^n P(x_j | y).$$

预测:

$$\hat{y} = \arg \max_y \left[\log P(y) + \sum_j \log P(x_j | y) \right].$$

Laplace smoothing:

$$P(x_j = v | y) = \frac{\text{count}(x_j = v, y) + \alpha}{\text{count}(y) + \alpha K}.$$

若 4 个 binary features 加 1 个 binary label: feature patterns 有 $2^4 = 16$ 种; feature-label 组合有 $2 \cdot 2^4 = 32$ 种; full joint 独立参数是 $2^5 - 1 = 31$ 。

9.2 Bayesian Network

BN 用 DAG 分解 joint distribution:

$$P(X_1, \dots, X_n) = \prod_i P(X_i | \text{Parents}(X_i)).$$

PPT 视角下, 一个 BN 包含三层意思:

- 图结构是 DAG; 每个 node 是一个 random variable。
- 每条 arc 表示 parent 对 child 的 direct influence。

- 每个 node 关联一个 conditional distribution/CPT, 即 $P(X_i | Parents(X_i))$ 。

BN 的好处是可以用 independence 和 conditional dependence 更紧凑地定义 joint distribution。给定 parents 后, 一个节点与其 non-descendants 条件独立。Markov blanket 包括 parents、children、children 的其他 parents。

Exact inference:

$$P(X | e) = \alpha P(X, e) = \alpha \sum_Y P(X, e, Y).$$

Variable elimination 通过局部 factors 相乘并逐个 sum out hidden variables, 通常比 naive enumeration 高效。

构建 BN/KB 的挑战: 手工写 KB 很繁琐。PPT 中把学习 BN 类比为两件事:

- **Structural learning:** 学 network structure, 也就是哪些变量之间有边。
- **Parameter estimation:** 学边和节点上的参数, 例如 CPT 中的概率值, 或某个概率密度函数的参数。

9.3 Bayes Rule 例题

给定

$$P(Boy) = 0.7, \quad P(Girl) = 0.3, \quad P(Short | Boy) = 0.9, \quad P(Short | Girl) = 0.2.$$

$$P(Girl | Short) = \frac{P(Short | Girl)P(Girl)}{P(Short | Boy)P(Boy) + P(Short | Girl)P(Girl)} = \frac{0.06}{0.69} = \frac{2}{23}.$$

9.4 Sampling

Prior sampling: 按 topological order 逐个采样。若 8 个样本中 $C = True$ 有 5 个:

$$\hat{P}(C) = 5/8.$$

Rejection sampling: 估计 $P(C | A, \neg D)$ 时只保留 evidence $A = True, D = False$ 的样本。若保留 3 个, 其中 2 个 $C = True$:

$$\hat{P}(C | A, \neg D) = 2/3.$$

Likelihood weighting: evidence 固定, non-evidence 正常采样, 每个样本权重

$$w = \prod_{E_i \in Evidence} P(E_i = e_i | Parents(E_i)).$$

若网络为 $A \rightarrow B \rightarrow C \rightarrow D$, 估计 $P(\neg A | B, \neg D)$, 则权重要乘

$$P(B | A) \cdot P(\neg D | C),$$

其中第一项看样本里的 A , 第二项看样本里的 C 。

Gibbs sampling: evidence variables 固定; 每一步只重采样一个 non-evidence variable; 相邻样本最多一个非 evidence 变量变化。

10 Evaluation, Tree 与 Generalization

10.1 Confusion Matrix

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \quad Precision = \frac{TP}{TP + FP},$$

$$Recall = \frac{TP}{TP + FN}, \quad F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}.$$

$$TPR = Recall, \quad FPR = \frac{FP}{FP + TN}.$$

ROC 横轴 FPR, 纵轴 TPR; AUC 衡量排序能力, 0.5 近似随机, 1.0 完美。

Project3 中 CTR 用 AUC, top-5 recommendation 用 nDCG@5:

$$DCG@5 = \sum_{j=1}^5 \frac{\mathbb{I}[i_j \in T_u^+]}{\log_2(j+1)}, \quad nDCG@5 = \frac{DCG@5}{IDCG@5}.$$

10.2 Generalization 与 Tuning

常见估计方法: random split、 k -fold cross-validation、bootstrap。报告多次实验最好写

$$mean \pm std.$$

Hyperparameter tuning 标准流程: training set 训练; validation set 选参数; test set 只做最终报告。

Training objective 不一定等于 performance metric。例如训练优化 cross-entropy, 但最终可能关心 F1、AUC 或 nDCG。

PPT 中特别强调: “accuracy=99% in 5 seconds” 这类说法风险很高, 因为测试环境和真实环境可能不同。内部测试只能尽量降低风险, 不能完全消除风险。Generalization performance 本身是 random variable, 所以需要多次 split/CV/bootstrap 来估计均值和方差。

常见 hyperparameters:

- SVM: kernel type、kernel parameters、regularization parameter C 。
- Neural network: hidden nodes、activation function、architecture、learning rate。
- Decision tree: branching factor、height/max depth。
- Clustering: number of clusters K 。

调参没有解析解, 因为每试一个 hyperparameter 都要重新估计 generalization performance。基础方法是 trial-and-error; 更高效的方法包括 heuristic search、black-box optimization, 或者用 surrogate evaluation 加速, 例如某些 kernelized linear regression 可以用 LOOCV 近似快速估计泛化误差。

10.3 Decision Tree

Entropy:

$$H(S) = - \sum_i p_i \log_2 p_i.$$

Information gain:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v).$$

Gini:

$$\text{Gini}(S) = 1 - \sum_i p_i^2.$$

防止 overfitting: 限制 max depth、minimum samples per leaf、pruning、bagging/random forest。

11 Unsupervised Learning

PPT 对 unsupervised learning 的定位: 人类学习不总是依赖监督信号; 现实中标注数据昂贵, 但未标注数据相对容易积累。它和 supervised learning 仍然共享

representation + algorithm + evaluation

三要素, 只是 unsupervised learning 的 evaluation/objective 往往更主观、更模糊, 因此问题更容易 ill-defined, 输出也可能偏离人的直觉。

Clustering 是 classification 在无监督场景下的对应问题: 给定数据, 把它们分成 K 个 groups, 使同一 cluster 内实例相似, 不同 clusters 之间不相似。但“相似”没有唯一标准; PPT 里强调, clustering 对很多 objective 都可能是 NP-hard, 本质上是在巨大 solution space 中搜索。

11.1 K-means

目标函数:

$$\min_{\{c_i\}, \{\mu_j\}} J = \min \sum_{i=1}^m \|x_i - \mu_{c_i}\|^2.$$

它等价于最小化 intra-cluster distance / within-cluster sum of squares。

迭代:

$$c_i = \arg \min_j \|x_i - \mu_j\|^2, \quad \mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i.$$

PPT 中的算法流程:

1. 预先给定 K 。
2. 随机初始化 K 个 cluster centers。
3. Assignment step: 把每个 instance 分到最近的 center。
4. Update step: 把每个 center 更新为该 cluster 内所有 instances 的 mean。
5. 重复 assignment/update, 直到 centers 不再变化或达到迭代上限。

特点与易错点:

- 不仅要定义 similarity metric, 通常还要能计算 average, 所以最自然适合 Euclidean feature vectors。
- 对初始化敏感, 只保证收敛到 local optimum; K-means++ 可以改善初始化。
- 对 outliers 敏感; 需要提前选择 K 。
- 对 graph data, 数据不一定是 feature vector。PPT 中提到 social network community detection 可以先把图嵌入到 Euclidean space, 也可以直接做 graph-cut/community detection。

11.2 PCA

目标: 给定 $n \times d$ 数据集, 用 linear transformation 映射到低维空间, 并尽量引入最小 information loss。PPT 中强调, 这两个条件会把 PCA 变成 eigenvalue decomposition 问题。

$$X_c = X - \bar{X}, \quad \Sigma = \frac{1}{m} X_c^T X_c, \quad Z = X_c W_k.$$

W_k 由前 k 个最大 eigenvalues 对应 eigenvectors 构成。

复习时可以这样写:

- PCA 找最大 variance directions。
- 用 covariance matrix 的 eigenvectors 作为 principal components。
- 选最大 eigenvalues 对应的前 k 个方向投影。
- 用途: visualization、noise reduction、compression。
- 局限: linear mapping, 不能展开明显非线性的 manifold。

11.3 LLE

Locally Linear Embedding 保留局部邻域结构:

1. 为每个点找 nearest neighbors。
2. 用 neighbors 线性重构该点, 求 weights。
3. 在低维空间中保持这些 reconstruction weights。

更完整的考试写法:

$$\min_{\{w_{ij}\}} \sum_i \left\| x_i - \sum_{j \in N(i)} w_{ij} x_j \right\|^2, \quad \sum_{j \in N(i)} w_{ij} = 1.$$

然后在低维表示 z_i 中解:

$$\min_{\{z_i\}} \sum_i \left\| z_i - \sum_{j \in N(i)} w_{ij} z_j \right\|^2.$$

直觉: 如果高维空间中一个点可以由邻居线性重构, 那么低维空间中也应保持同样的局部重构关系。LLE 比 PCA 更适合非线性 manifold, 但依赖 neighborhood size, 且对噪声和采样密度敏感。

Project2 经验: PCA/LLE 能提速或降噪, 但如果 hidden ground truth 与原距离度量一致, 过强降维可能改变 ranking, 需用 validation/proxy metric 判断。

12 Recommender Systems 与 Knowledge Graphs

12.1 推荐系统基础

目标：学习

$$score(user, item).$$

PPT 中把 recommender system 的核心说得很直接：推荐系统本质上估计 user 与 item 之间发生 interaction 的概率。输入是 user 和 item，输出 score，score 越高表示用户越可能感兴趣。

类型：

- Content-based: 关注 item 的 side information/attributes/features，为每个用户学习偏好模型，再推荐与用户历史行为相似的 items。
- Collaborative filtering: 只依赖 historical user-item interaction records，通过其他 users/items 的行为来预测偏好。
- Hybrid: 融合多种信号。

挑战: cold start、sparsity、scalability、accuracy-efficiency trade-off。实际系统可能有百万级 users/items，因此 score function 既要准，也要能在秒级完成推荐。

图视角：

- Interaction records 可以看成 user-item bipartite graph。
- 推荐就是预测 user 和 item 之间是否应有一条 edge。
- 如果加入 KG，interaction graph 与 heterogeneous directed KG 可以合成一个更大的 mixed graph。

12.2 Pearson Similarity

$$sim(u, v) = \frac{\sum_i (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_i (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_i (r_{v,i} - \bar{r}_v)^2}}.$$

它衡量 normalized covariance，可用于 user-user 或 item-item neighbor recommendation。

基于 correlation 的表示：

- 一个 user/item 可以被表示成它与所有其他 users/items 的 correlation vector。
- 优点: interpretability 高，容易解释“为什么推荐”。
- 缺点: scalability 差；当 users/items 很多时，向量高维、稀疏且昂贵。

简单 score function 例子：

$$f(u, i) = \sum_{u' \in U} c_{u,u'} r_{u',i}, \quad f(u, i) = \sum_{i' \in M} c_{i,i'} r_{u,i'}.$$

前者是 user-user CF，后者是 item-item CF。

12.3 Matrix Factorization

$$R \approx PQ^T, \quad P \in \mathbb{R}^{n \times d}, \quad Q \in \mathbb{R}^{m \times d}.$$

预测:

$$\hat{r}_{u,i} = P_u Q_i^T.$$

训练目标:

$$\min_{P,Q} \sum_{(u,i) \in R'} (r_{u,i} - P_u Q_i^T)^2.$$

其中 R' 是已知 interaction records 的集合, 因为完整 R 通常不可观测。 P_u 和 Q_i 是低维 embedding vectors。

如果不用简单 dot product, 也可以定义更复杂的 interaction probability function:

$$\hat{r}_{u,i} = f(P_u, Q_i; M),$$

其中 M 可以是 MLP 等模型。矩阵乘法是最简单、最高效、最常考的形式。

Project3 的 MF 打分还可带 bias:

$$s_{\text{mf}}(u, i) = p_u^T q_i + b_u + b_i + b_0.$$

BCE:

$$\mathcal{L}_{BCE} = -\frac{1}{|\mathcal{B}|} \sum_{(u,i,y)} [y \log \sigma(s) + (1-y) \log(1 - \sigma(s))].$$

BPR:

$$\mathcal{L}_{BPR} = \frac{1}{|\mathcal{B}^+|} \sum_{(u,i^+,i^-)} \log(1 + \exp(s(u, i^-) - s(u, i^+))).$$

12.4 Knowledge Graph

KG 是更 practical 的知识表示方式。PPT 中的核心定义: Knowledge Graph 是 large-scale semantic network, 用 vertices 和 edges 图形化表示知识。

- Vertices: entities 和 concepts。
- Edges: relations 和 properties。
- KG 通常是 heterogeneous directed graph。

为什么需要 KG:

- 大部分人类知识以自然语言表达, 但很多应用更关心 entities 及其 relationships。
- 相比完整逻辑 KB, KG 缩小 commonsense knowledge 的边界, 更容易落地。
- KG 可用于 recommender system、Q&A system、information retrieval 等。

KG 存储 RDF-style triples:

$$(head, relation, tail), \quad \langle h, r, t \rangle.$$

其中 head entity 是 subject, relation type 是关系类别, tail entity 是 object。

12.4.1 KG Construction

构建来源: structured databases、information extraction、人工领域知识。PPT 给出的半自动构建流程:

1. Human experts 写 rules/templates 或 dictionaries。
2. 从 corpus 中识别 entities, 得到 vertex set。
3. 做 relation extraction, 得到 edge set。
4. 进行 automatic annotation 或人工校正。
5. 对已有 KG 做 completion, 补全缺失关系。

Automatic entity recognition:

- 统计方法: TF-IDF。如果一个词在当前文档频繁出现、在其他文档不常出现, 更可能是 meaningful entity。
- Entropy。如果一个词的左右邻居很多样, entropy 高, 更可能是 meaningful entity:

$$H(u) = - \sum_{x \in \mathcal{X}} p(x) \log p(x).$$

这里 $p(x)$ 可以表示某个 left/right neighbor word 出现的概率。

- 机器学习方法: NER, 把 entity recognition 建模为 sequence labeling。典型标签是 BIO, 例如 B-People, I-People, O, B-Contest。

Automatic relation extraction:

- 输入: 包含两个 entities 的句子。
- 输出: 句子表达的 relation category。
- 可视作 supervised text classification, 常用 RNN、Transformer 等。

12.4.2 Knowledge Graph Completion

KGC 的目标是在已有 KG 中寻找 missing relations。需要 KGC 的原因:

- 完整 rule base 太贵。
- relation extraction 不能抽出所有关系。
- 原始文本本身包含的关系信息也不完整。

两类方法:

- Path-based methods: 把两个 entities 之间的 relation-type sequence 当作 feature。比如一条路径可写为

$$\langle \text{work at, locate at, belongs to} \rangle,$$

它可以提示某种隐含关系。

- Embedding-based methods: 把 entities 和 relations 表示为低维向量, 设计 score function $g(\mathbf{h}, \mathbf{r}, \mathbf{t})$ 。分数越高, triple (h, r, t) 越可能为真。

常见训练思想:

$$g(\mathbf{h}, \mathbf{r}, \mathbf{t}) \text{ for true triples} > g(\mathbf{h}', \mathbf{r}', \mathbf{t}') \text{ for corrupted/false triples.}$$

答题时不必死记具体损失, 重点说清楚“真 triple 高分, 假 triple 低分”。

12.4.3 KG-Based Recommender System

Project3 的复习表达:

$$\text{final score} = \text{popularity} + \text{temporal prior} + \text{KG content} + \text{item CF} + \text{MF}.$$

这体现 hybrid recommendation: KG 缓解 sparsity, CF 捕捉协同偏好, MF 学 latent representation。

PPT 中的 KG-based RS 图视角:

- Interaction records 是 bipartite graph。
- KG 是 heterogeneous directed graph。
- 两者可以融合为 mixed graph。
- 从 mixed graph 中可得到 paths between users and items、correlations among users/items、embeddings of users/items。
- GNN 是典型方法: 每个节点有 initial embedding, 最终 embedding 由 neighborhood embeddings 聚合得到, 再用 MLP 或 matrix multiplication 计算 $f(u, i)$ 。

答题句式: 知识图谱题常见答法: KG 是 heterogeneous directed semantic graph; RDF triple 写成 $\langle head, relation, tail \rangle$; construction 包括 entity recognition、relation extraction 和 completion; KGC 可用 path-based 或 embedding-based missing relation prediction。

13 PPT 强调的概念问答

这一节整理 PPT 中反复出现的“What/Why/How”型问题。它们不一定是手算题, 但很适合出简答题、解释题或 project-style 综合题。

13.1 Search 与 Problem-Specific Search

1. What is search?

Search 是从很多 possible options 中找一个 satisfactory/user-defined option。AI 中很多思考过程都可视为 search。

2. 为什么不能简单 enumerate and compare?

因为 state/solution space 往往巨大, 枚举所有 options 成本太高。因此需要 organized search process, 用 frontier/search tree 或 solution-space operator 有选择地探索。

3. What is a good search method?

四个标准: completeness、optimality、time complexity、space complexity。复杂度通常由 branching factor b 、最浅目标深度 d 、最大深度 m 等决定。

4. 为什么需要 heuristic search?

因为 cost is an issue。Heuristic search 利用 problem-specific knowledge 给 tree search 加 bias, 不再均匀枚举所有可能。

5. 如何设计 admissible heuristic?

PPT 强调两条思路: relaxed problem 和 sub-problem。Relaxed problem 去掉部分约束, 得到的最优代价不会超过原问题最优代价; sub-problem 只看原问题一部分, 也常给出 lower bound。

6. 实践中为什么 completeness/optimality 没有课上那么强调?

很多实际问题中 admissible heuristic 难设计; 即使能保证 optimality, 时间复杂度也可能不可承受。所以工程上常更看重 acceptable solution under budget。

7. 拿到一个 search problem 先问什么?

PPT 总结:

- 如何表示 search space: state-space tree 还是 solution space?
- Objective function 和 constraints 是什么?
- 用 tree search、heuristic search, 还是 hill climbing/SA/GA 等 solution-space search?
- Search operator、evaluation criterion、replacement strategy 怎么定义?
- 有没有 problem-specific knowledge 可加入 operator 或 heuristic?

13.2 Beyond Classical Search 与 Evolutionary Search

1. Classical search 和 solution-space search 的区别?

Classical search 通常用 tree representation, 每个节点是 partial decision/state; solution-space search 中每个元素就是 complete candidate solution。

2. Local search 的两个核心组件是什么?

Search operator: 如何生成新候选解; evaluation/replacement criterion: 如何决定是否接受新解。

3. Simulated Annealing 相比 hill climbing 解决什么问题?

Hill climbing 只接受更好解, 容易卡在 local optimum; SA 在高温时可按概率接受更差解, 温度降低后逐渐变贪心。

4. Tabu Search 的 key idea 是什么?

Incorporate memory: 用 tabu list 避免短期内访问重复解或重复操作。挑战是如何定义 tabu list。

5. Bayesian Optimization 的 key idea 与限制?

Key idea: build a model to guess which solution is good。限制: 模型构建不平凡, 可能需要较多数据, 通常更适合 low-dimensional problems。

6. 为什么 population-based search 可能有益?

Population 同时维护多个 candidate solutions, 相当于在寻找一个较好的 sampling distribution, 有 smoothing effect, 能降低单点搜索过早陷入局部最优的风险。

13.3 Machine Learning 与 Evaluation

1. What is learning in AI?

Given observations/data from environment, an agent improves its agent function。课程总结为: learning is search for hypothesis/functions that generalize well。

2. Machine learning 的四个 key questions?

- Data 的 format 是什么? 即 data representation。
- Agent function/model 长什么样? 即 model representation。
- 如何衡量 improvement? 即 objective/evaluation function。
- 用什么 learning algorithm 找到好的 function?

总式:

$$\text{Representation} + \text{Algorithm} + \text{Evaluation} = \text{Model}.$$

3. 为什么 performance evaluation crucial?

因为模型和训练算法之外, 用户真正关心的是部署环境中的表现。一个 “accuracy 99%, 5 seconds” 的说法可能有三类风险: metric 不符合业务需求; 数据只部分反映现实; 运行硬件或部署环境不同。

4. 选择 objective function 的两个原则?

第一, 与 user requirements consistent; 第二, 最好有 existing easy-to-use algorithm 可以优化它。若目标和优化代理不一致, 即使代理指标提升, 真实任务也可能无改善。

5. 为什么 generalization performance 是 random variable?

因为 train/test split、采样数据、随机初始化、训练过程都会影响估计结果。应通过 random split、cross-validation、bootstrap 多次评估, 报告 average 和 standard deviation。

6. 为什么 hyperparameter tuning 没有解析解?

每个 hyperparameter 的好坏都要训练模型并估计 generalization performance 才知道, 所以本质上是 trial-and-error 或 black-box optimization。

13.4 SVM, Neural Network 与 Unsupervised Learning

1. 为什么实际中常用 soft-margin SVM?

即使用 kernel trick, 也很难保证训练数据完全 linearly separable; soft margin 允许 margin violation, 用 C 平衡 margin 和 training errors。

2. 为什么早期 deep MLP 难训练?

PPT 提到多隐藏层 fully connected NN 很难训练, 历史上常见问题包括优化困难、梯度传播问题、参数多导致过拟合。现代方法通常依赖更好的 activation、initialization、normalization、optimizer 和 architecture。

3. Unsupervised learning 和 supervised learning 最大区别?

二者都需要 representation、algorithm、evaluation; 但 UL 没有明确 ground truth, evaluation/objective 更 subjective/vague, 所以更容易 ill-defined。

4. 为什么 clustering 是 ill-defined?

因为 “similarity” 和 “cluster quality” 没有唯一标准。不同 metric/objective 会得到不同分组；很多 clustering objective 的 solution space 很大，求全局最优困难。

5. PCA 和 LLE 的概念区别?

PCA 是 linear dimensionality reduction，寻找最大方差方向；LLE 是 nonlinear manifold learning，保持每个点由近邻线性重构的局部关系。

13.5 Logic, FOL, BN 与 KG

1. 为什么 knowledge representation 对 AI 重要?

人类不只基于 instinct/program 行动，也会积累 knowledge，并通过 inference 从已知推出未知。AI agent 也需要 represent, store, exploit knowledge。

2. Logic 在知识表示中的作用?

Logic 是 formal language for representing knowledge。Inference 是从已有 sentence 推出新 sentence 的过程；model checking 和 theorem proving 是两类基本推理思想。

3. CSP 和 SAT 有什么关系?

CSP 的变量、domains、constraints 可以编码为 propositional variables 和 clauses，从而转化为 SAT/CNF 推理问题。直觉上，SAT 是 binary-domain CSP 的一种核心形式。

4. 为什么需要 FOL?

Propositional logic 对对象、关系和量词表达不够自然；FOL 更接近自然语言，可用 objects、predicates、functions、quantifiers 表示结构化世界，KB 构建有时更方便。

5. 为什么需要概率表示状态?

现实世界 partially observable，agent 很少精确知道自己处于哪个 state；因此要用 probability 表示 uncertainty，并结合 utility 做 rational decision。

6. What is a Bayesian Network?

BN 是 DAG：node 是 random variable，每个 node 带 CPT/conditional distribution，edge 表示 parent 对 child 的 direct influence。BN 用条件独立紧凑表示 joint distribution。

7. 构建 BN 或 KB 的挑战是什么?

手工写 KB 很 tedious，甚至 unaffordable。可从 raw data 中学习：structural learning 学网络结构，parameter estimation 学 CPT 或概率分布参数。

8. 为什么需要 Knowledge Graph?

完整逻辑 KB 很难写，也难表达 hidden knowledge。KG 缩小 knowledge 的边界，用 entities 和 relations 表示知识，更 practical，适合 RS、QA、IR 等应用。

9. KG 和 logic KB 的区别?

Logic KB 更 rigorous，适合严格推理；KG less rigorous but more practical，用 heterogeneous directed graph/RDF triples 存储实体关系，便于从文本和结构化数据中构建。

10. 为什么 KG-based recommender 需要 side information?

核心数据是 user-item interactions，但 interaction sparse、cold-start 严重。KG 提供 item/entity side information，可和 interaction graph 融合成 mixed graph，帮助学习 user/item features 或 embeddings。

14 Genetic Algorithm 与 Fair Comparison

GA 流程:

1. Encode individual/chromosome。
2. Initialize population。
3. Compute fitness。
4. Select parents。
5. Crossover 产生 offspring。
6. Mutation。
7. Survivor selection。
8. 达到收敛、代数或时间预算后停止。

GA vs backpropagation 训练神经网络的公平比较:

1. 相同 dataset split。
2. 相同 neural network architecture。
3. 相同 preprocessing 和 input features。
4. 相同 evaluation metric。
5. 相同 compute 或 wall-clock budget。
6. 两者都在 validation set 调参。
7. 多 random seeds, 报告 mean/std。
8. untouched test set 只做最终一次评价。

答题句式: For GA, one individual is the flattened vector of all network weights. Fitness is validation accuracy or negative validation loss. Backprop trains the same architecture using SGD/Adam under the same budget.

15 Reinforcement Learning

核心元素:

$$s, a, r, \pi(a \mid s), \gamma, V(s), Q(s, a).$$

Return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Bellman optimality:

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V^*(s')].$$

Q-learning update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

ϵ -greedy:

$$\begin{cases} \text{random action,} & \epsilon \\ \arg \max_a Q(s, a), & 1 - \epsilon. \end{cases}$$

16 考前最后检查清单

1. 会按 tie-breaking 画 BFS/A* 搜索树。
2. 会计算 $f(n) = g(n) + h(n)$ 。
3. 会求 admissible 的边权 c 范围。
4. 会解释 local search、SA、tabu。
5. 会填 alpha/beta 并标剪枝。
6. 会跑 AC-3, 写 domain 变化。
7. 会把 $p \rightarrow \neg(p \vee q)$ 转成 $\neg p$ 。
8. 会自然语言建模并转 CNF。
9. 会 DPLL unit propagation。
10. 会 FOL resolution 得 contradiction。
11. 会写 perceptron update。
12. 会推 logistic gradient。
13. 会写 hard/soft-margin SVM。
14. 会解释 kernel trick 和 C 。
15. 会用 smoothing 做 Naive Bayes。
16. 会算 $P(Girl | Short)$ 。
17. 会 exact inference 与 VE。
18. 会 prior/rejection/LW/Gibbs。
19. 会算 accuracy/precision/recall/F1。
20. 会解释 ROC/AUC/threshold。
21. 会 entropy/information gain。
22. 会 K-means assignment/update。
23. 会 PCA 与 LLE 流程。
24. 会写 $R \approx PQ^T$ 。
25. 会解释 KG triples/NER/RE/KGC。
26. 会写 GA 公平比较方案。
27. 会 Bellman 与 Q-learning。

来源说明

本讲义综合整理自 AI Notes.md、ppts/AI Review(1).pdf、ppts/Exam Paper (3).pdf、ppts/Logic and Bayesian Questions(1).pdf、ppts/AI_Lec06-Lec12、Labs 1-11 以及 Projects 1-3 的报告与代码材料。